ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1

«Організація професійного Python-проєкту та Git»

з курсу «Професійний Python»
Варіант №1 — «Система обліку студентів»

Виконала: Руссу Анастасія Сергіївна

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 25.10.2026

Львів — 2026

1. Мета роботи

Навчитися організовувати Python-програму як окремий проєкт: підготувати віртуальне середовище, розмістити пакет у каталозі src, відокремити модель даних від обчислень і консольного інтерфейсу. Закріпити роботу з pyproject.toml, документацією та Git під час реалізації системи обліку студентів.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Система обліку студентів». Необхідно зберігати для кожного студента ім’я, прізвище, групу та середній бал. Програма повинна додавати записи, показувати список, знаходити студентів заданої групи, визначати студента з найбільшим середнім балом і розраховувати середній бал групи.

Відповідно до методички реалізовано три основні модулі, модель даних, анотації типів і окрему точку входу. Додатково виконано введення з клавіатури, консольне меню, валідацію, сортування та використано dataclass. Для оцінок прийнято шкалу від 0 до 100; у методичці конкретну шкалу не задано. Записи зберігаються в пам’яті протягом одного запуску.

3. Структура проєкту

```
student_manager/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── student_manager/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       └── services.py
└── tests/
    └── test_services.py
```

У src/student_manager розміщено код пакета, а в tests — перевірки його функцій. Файл __init__.py позначає звичайний Python-пакет і містить номер версії. Каталоги .venv та .git створюються під час налаштування середовища і репозиторію; до вихідного коду вони не належать.

4. Опис створених модулів

4.1. models.py

Клас Student об’єднує чотири характеристики студента. Декоратор dataclass створює ініціалізатор за оголошеними полями. У __post_init__ прибираються крайні пробіли, назва групи переводиться у верхній регістр і перевіряються введені дані. Властивість full_name потрібна для виведення прізвища та імені в одній колонці.

```
from dataclasses import dataclass

from math import isfinite
```

```
@dataclass

class Student:

    first_name: str
    last_name: str
    group: str
    average_grade: float


    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip().upper()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError(
                "Ім'я, прізвище та група не можуть бути порожніми."
            )
        if (not isfinite(self.average_grade)
                or not 0 <= self.average_grade <= 100):
            raise ValueError("Середній бал має бути числом від 0 до 100.")


    @property
    def full_name(self) -> str:
        return f"{self.last_name} {self.first_name}"
```

Перевірка isfinite відхиляє NaN та нескінченність, які float може прочитати як числові значення. Якщо поле порожнє або бал виходить за межі шкали, створення об’єкта завершується ValueError. Некоректний запис не потрапляє до списку.

4.2. services.py

Цей модуль працює зі списком об’єктів Student і не викликає input або print. Додавання змінює початковий список. Пошук формує окремий список студентів потрібної групи, тому записи інших груп не беруть участі в обчисленні її середнього бала.

```
def add_student(students: list[Student], student: Student) -> None:
    students.append(student)


def filter_by_group(students: list[Student], group: str) -> list[Student]:
    group = group.strip().upper()
    return [student for student in students if student.group == group]
```

Для визначення найкращого студента послідовно порівнюються середні бали. Змінна best містить найкращий знайдений запис. За однакових максимальних значень залишається перший доданий студент. Порожньому списку відповідає результат None.

```
def find_best_student(students: list[Student]) -> Student | None:
    if not students:
        return None
    best = students[0]
    for student in students[1:]:
        if student.average_grade > best.average_grade:
            best = student
    return best
```

Середній бал групи дорівнює сумі середніх балів її студентів, поділеній на кількість знайдених записів. Якщо групи немає, повертається None: це дозволяє відрізнити відсутність даних від справжнього нульового середнього бала. Функція рейтингу повертає новий список і не змінює порядок початкових записів.

```
def calculate_group_average(
    students: list[Student], group: str,
) -> float | None:
    selected = filter_by_group(students, group)
    if not selected:
        return None
    total = sum(student.average_grade for student in selected)
    return total / len(selected)


def sort_by_grade(students: list[Student]) -> list[Student]:
    return sorted(students, key=lambda student: student.average_grade,
                  reverse=True)
```

4.3. main.py

Модуль відповідає за взаємодію з користувачем. Функція read_student зчитує чотири поля, перетворює бал на float і створює Student. Десятковий роздільник можна записувати крапкою або комою. У main перехоплюється ValueError, після чого користувач повертається до меню.

```
def read_student() -> Student:
    first_name = input("Ім'я: ")
    last_name = input("Прізвище: ")
    group = input("Група: ")
    try:
        grade = float(input("Середній бал (0–100): ").replace(",", "."))
    except ValueError:
        raise ValueError("Середній бал потрібно ввести числом.") from None
    return Student(first_name, last_name, group, grade)
```

Під час запуску main створюється порожній список. Цикл обробляє вибраний пункт, викликає відповідну функцію services і виводить результат через print_students. Нижче наведено фрагмент обробки додавання запису; інші пункти меню викликають пошук, розрахунок середнього бала, визначення найкращого студента та сортування.

```
if command == "1":
    try:
        student = read_student()
        add_student(students, student)
        print(f"Додано: {student.full_name}.")
    except ValueError as error:
        print(f"Помилка: {error}")
```

Функції можна імпортувати для перевірки без запуску меню. Виклик main виконується лише за умови, що модуль є точкою входу програми:

```
if __name__ == "__main__":
    main()
```

4.4. test_services.py

Тести перевіряють додавання, фільтрацію, середній бал лише вибраної групи, максимальний бал та рейтинг. Окремо перевірено порожній список, відсутню групу, рівні максимуми, порожні поля, бали 0 і 100, вихід за межі шкали, NaN та нескінченність. Вигадані дані задано у фікстурі тестів; у самій програмі готових студентів немає.

```
def test_average_of_selected_group(students: list[Student]) -> None:
    assert calculate_group_average(students, "ФЕП-32") == pytest.approx(89.0)
    assert calculate_group_average(students, "ФЕП-31") == pytest.approx(95.0)
    assert calculate_group_average(students, "ФЕП-99") is None
    assert calculate_group_average([], "ФЕП-32") is None
```

Після встановлення пакета командою python -m pytest -q виконано 14 перевірок. Усі вони завершилися успішно. Додатковими запусками меню перевірено нечисловий бал, невідомий пункт і завершення роботи через обидві команди запуску.

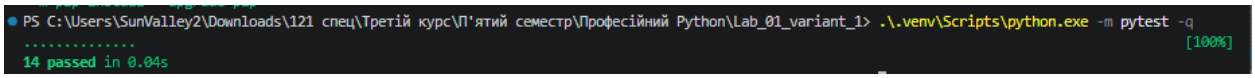


Рис. 1. Результат виконання автоматизованих тестів

5. Файл pyproject.toml

У файлі описано спосіб збирання та параметри встановлення пакета. Секція build-system задає setuptools як backend. У project зазначено назву, версію, авторку, опис, README та мінімальну версію Python. Залежність pytest потрібна лише для тестів, тому винесена в додаткову групу test.

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "student-manager"
version = "0.1.0"
description = "Console student register for laboratory work 1"
readme = "README.md"
requires-python = ">=3.11"
authors = [{name = "Руссу Анастасія"}]

[project.optional-dependencies]
test = ["pytest>=8.0"]

[project.scripts]
student-manager = "student_manager.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

У project.scripts команда student-manager пов’язана з функцією main. Налаштування packages.find вказує шукати пакети в src. У конфігурації pytest зазначено каталог tests; pythonpath не підміняється, тому тести перевіряють імпорт установленого пакета. Застосунок не потребує сторонніх бібліотек під час звичайної роботи.

6. Файл .gitignore

Середовище, кеші та результати збирання створюються локально й не повинні потрапляти в історію. Файл .env також виключено, оскільки він може містити приватні налаштування. Вміст .gitignore:

```
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

7. Фрагмент README.md

README містить призначення програми, вимоги, встановлення для різних оболонок, команди запуску й тестування, опис модулів та приклад введення. У ньому також пояснено обробку помилок і те, що після виходу введені записи не зберігаються. Нижче наведено фрагменти документації проєкту.

```
# Система обліку студентів

Лабораторна робота №1 з курсу «Професійний Python», варіант №1.
Авторка: Руссу Анастасія, ФЕП-32. Викладач: Свелеба С. А.

## Приклад роботи

Натисніть `1` і введіть чотири значення:

Ім'я: Марія
Прізвище: Дорош
Група: ФЕП-32
Середній бал (0–100): 86,5
Додано: Дорош Марія.

## Тести та Git

python -m pytest -q
git status
git log --oneline
```

8. Створення virtual environment та запуск

Роботу виконано в послідовності пункту 25 методички. Спочатку перевірено Python, створено каталог student_manager і середовище .venv, активовано його та ініціалізовано Git. Далі додано .gitignore, каталог src, пакет і модулі з анотаціями типів. Після налаштування pyproject.toml пакет установлено в editable mode, перевірено запуск, підготовлено README й завершено історію трьома комітами.

Фактична перевірка виконана в Linux із Python 3.12.14. Використано такі команди створення та активації середовища:

```
python --version
mkdir student_manager
cd student_manager
python -m venv .venv
source .venv/bin/activate
git init -b main
```

Після створення файлів проєкту виконано встановлення, перевірку пакетів, запуск і тести:

```
python -m pip install -e ".[test]"
python -m pip list
python -m student_manager.main
python -m pytest -q
```

Параметр -e пов’язує встановлений пакет із вихідним кодом у src. Після редагування Python-файлів повторне встановлення не потрібне. Частина [test] додатково встановлює pytest. У списку пакетів після встановлення присутній student-manager версії 0.1.0.

У Windows PowerShell замість source використовується .\.venv\Scripts\Activate.ps1, а в cmd — .venv\Scripts\activate.bat. Решта команд однакова. Також перевірено запуск через console script:

```
student-manager
```

9. Результат роботи програми

Для перевірки введено три вигадані записи: Марія Дорош — ФЕП-32, 86,5; Данило Левицький — ФЕП-31, 95; Соломія Климчук — ФЕП-32, 91,5. Значення введено через меню, вони не закодовані в застосунку. На рисунках показано знімки відображення фактичного консольного журналу запуску; повторні блоки меню між операціями опущено.

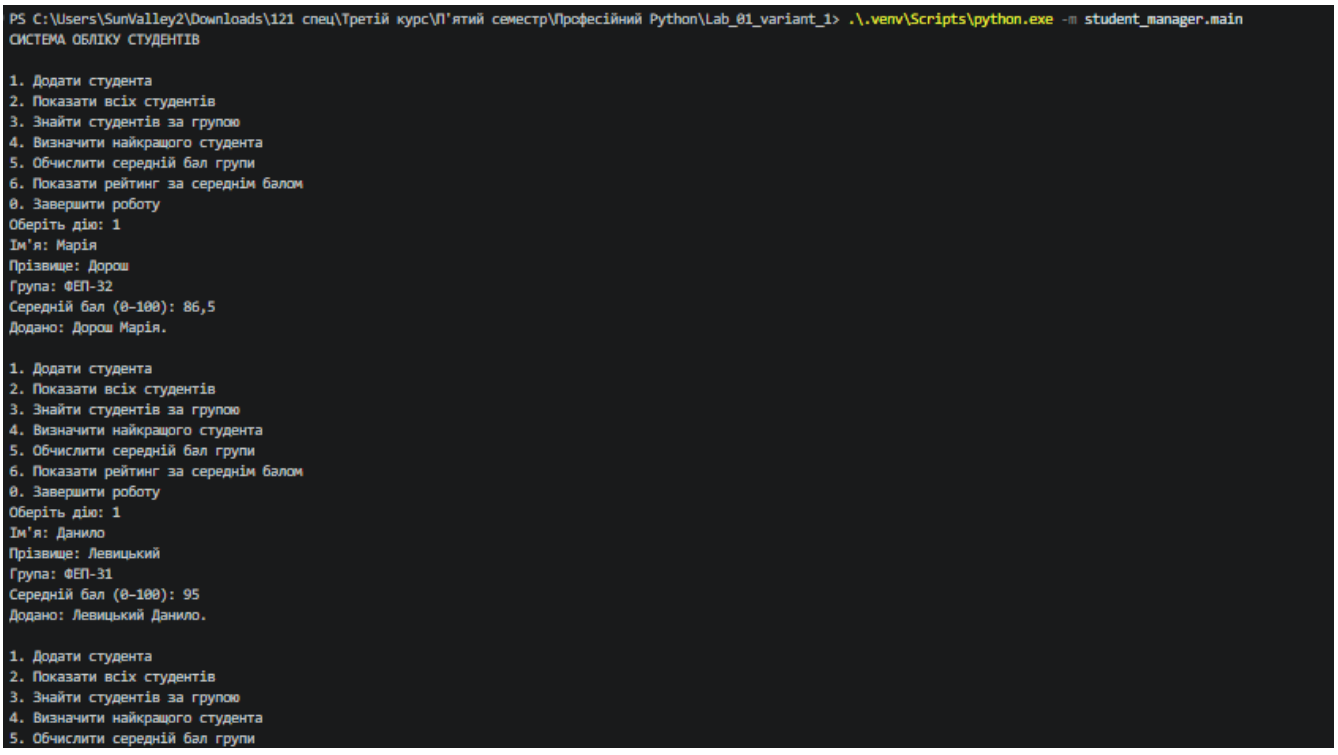


Рис. 2. Запуск меню та додавання студентів

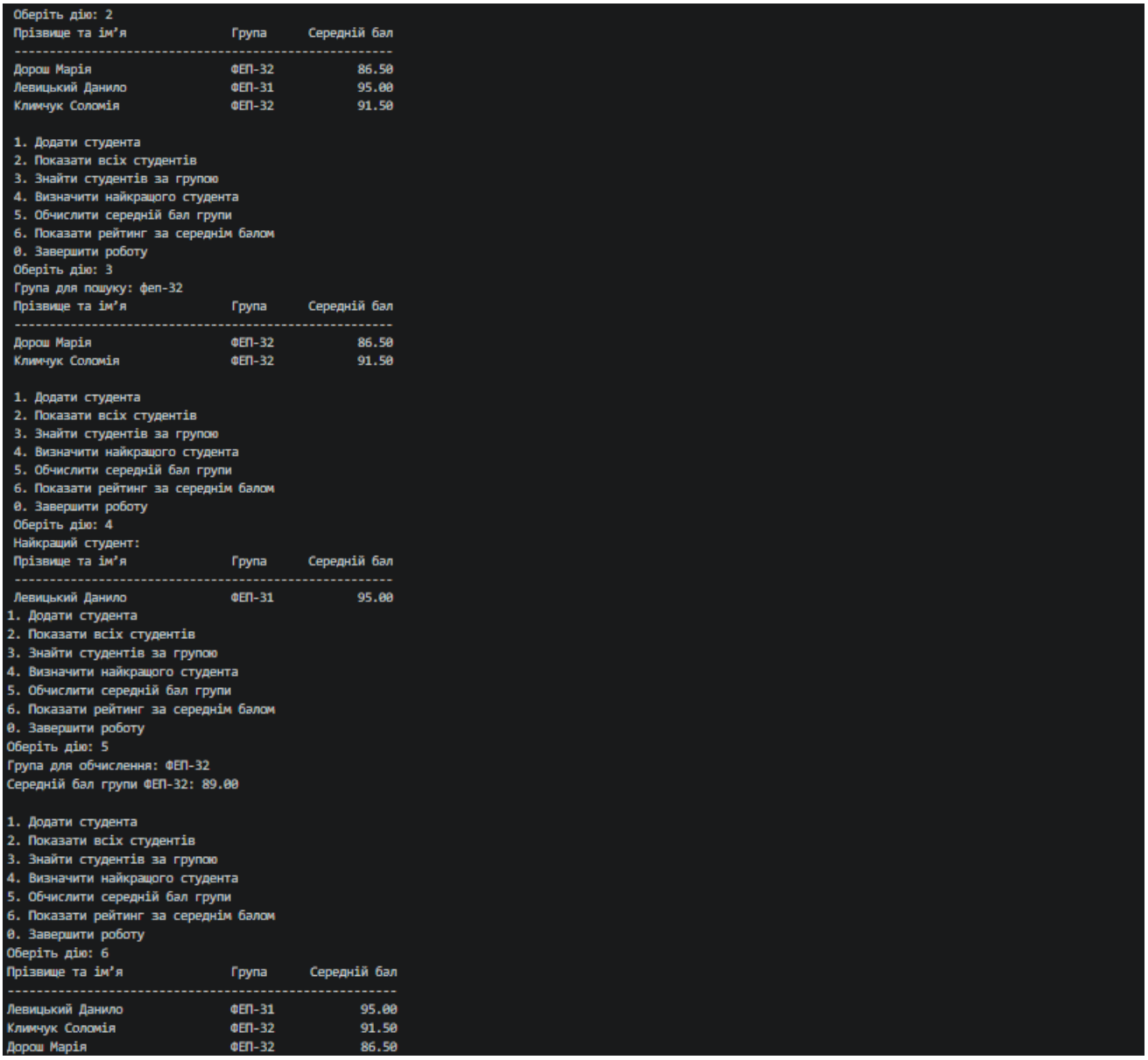


Рис. 3. Список, пошук за групою, найкращий студент і рейтинг

Пошук феп-32 повернув Марію Дорош і Соломію Климчук незалежно від регістру назви групи. Середній бал цієї групи становить (86,5 + 91,5) / 2 = 89,00. Данило Левицький має найбільший бал серед усіх записів — 95,00; його оцінка не впливає на результат ФЕП-32. Рейтинг сформовано у порядку 95,00; 91,50; 86,50.

Для групи ФЕП-99 даних немає, тому програма виводить відповідне повідомлення. Спроба додати студента з балом 105 відхиляється: після помилки меню продовжує працювати, а новий запис не додається.



Рис. 4. Обробка відсутньої групи та некоректного бала

10. Робота з Git

Репозиторій створено локально. Зміни збережено трьома логічними етапами: структура і конфігурація; модель, операції та тести; консольне меню й документація. Коміти створено під час підготовки відповідних файлів, а наведені нижче ідентифікатори отримано з цього репозиторію.

```
git add .gitignore pyproject.toml src/student_manager/__init__.py
git commit -m "Prepare src layout and package configuration"

git add src/student_manager/models.py src/student_manager/services.py tests
git commit -m "Implement student operations and boundary tests"

git add src/student_manager/main.py README.md
git commit -m "Add interactive menu and usage instructions"
```

10.1. Результат git status

```
On branch main
nothing to commit, working tree clean
```

Робоче дерево чисте: усі зміни вихідного коду зафіксовано. Віртуальне середовище, кеші та egg-info не відстежуються завдяки .gitignore.

10.2. Результат git log --oneline

```
4755e3e Add interactive menu and usage instructions
f025623 Implement student operations and boundary tests
d74b937 Prepare src layout and package configuration
```

11. Відповіді на контрольні запитання

1. Що таке Python-проєкт?

Це файли, потрібні для розробки та запуску програми: вихідний код, конфігурація, опис залежностей, документація і, за потреби, тести. У цій роботі вони об’єднані в каталозі student_manager.

2. Чим проєкт відрізняється від одного Python script?

Скрипт зазвичай виконує завдання в межах одного файла. У проєкті код розподілено між пов’язаними модулями, а встановлення та роботу описано окремими файлами.

3. Що таке Python module?

Модуль — одиниця Python-коду, яку можна імпортувати. У цьому проєкті це файли models.py, services.py та main.py.

4. Що таке Python package?

Пакет об'єднує модулі під спільним ім'ям. Наприклад, student_manager.services звертається до services усередині пакета student_manager.

5. Для чого потрібний __init__.py?

Він позначає звичайний пакет і виконується при його першому імпорті. Тут можна визначити версію або спільні імпорти. Існують також namespace-пакети без цього файла.

6. Що таке src-layout?

Це розміщення пакетів програми в окремому каталозі src. README, конфігурація та тести при цьому залишаються поза каталогом пакета.

7. Які переваги має src-layout?

Він відокремлює код пакета від допоміжних файлів і зменшує ризик випадкового імпорту коду з кореня проєкту замість установленого пакета. Це допомагає помітити помилки пакування.

8. Для чого використовується virtual environment?

Для кожного проєкту можна мати власний набір бібліотек та їхніх версій. Встановлення залежностей у .venv не змінює пакети інших середовищ.

9. Як створити .venv?

У каталозі проєкту виконують python -m venv .venv. Назва .venv є прийнятою домовленістю; каталог можна назвати й інакше.

10. Як активувати virtual environment?

У PowerShell: .\.venv\Scripts\Activate.ps1; у cmd: .venv\Scripts\activate.bat; у Linux/macOS: source .venv/bin/activate. Активація додає каталог виконуваних файлів середовища до PATH.

11. Для чого використовується pip?

pip установлює, оновлює та видаляє Python-пакети. Наприклад, python -m pip list показує пакети, доступні вибраному інтерпретатору.

12. Чому рекомендується python -m pip?

Так pip запускається через конкретний інтерпретатор python. Це зменшує ймовірність установити бібліотеку до іншої версії Python або іншого середовища.

13. Для чого потрібний pyproject.toml?

Цей файл описує систему збирання, метадані та залежності проєкту. У ньому також можна зберігати параметри інструментів, зокрема pytest.

14. Що містить секція [project]?

Основні відомості про пакет: name, version, description, requires-python, authors, readme та залежності. Набір полів залежить від потреб конкретного проєкту.

15. Що таке editable installation?

Це встановлення через pip install -e, за якого імпорт пов’язаний із робочим вихідним кодом. Зміни Python-файлів одразу враховуються при наступному запуску; зміни метаданих можуть потребувати повторного встановлення.

16. Для чого використовується if __name__ == "__main__"?

Умова дозволяє викликати main при запуску файла або через python -m. При звичайному імпорті модуля його ім’я інше, тому меню автоматично не відкривається.

17. Що таке Git?

Git зберігає версії файлів і зв’язки між ними. Це розподілена система: локальний репозиторій містить історію, з якою можна працювати без постійного доступу до сервера.

18. Що таке repository?

Це сховище об’єктів та історії Git. У звичайному локальному проєкті службові дані містяться в .git, а поряд розташовані файли робочого дерева.

19. Що таке commit?

Коміт фіксує стан підготовлених змін, автора, повідомлення та зв’язок із попередньою історією. Його можна визначити за унікальним хешем.

20. Для чого використовується git status?

Команда показує поточну гілку, підготовлені й непідготовлені зміни та невідстежувані файли. Перед комітом вона допомагає перевірити, що саме буде зафіксовано.

21. Для чого git add?

Команда переносить вибрані зміни до індексу — підготовленого стану для наступного коміту. Якщо файл знову змінити після git add, нові зміни потрібно додати окремо.

22. Для чого git commit?

Вона створює коміт із поточного індексу. Самостійно ця команда не публікує файли на GitHub і не додає всі непідготовлені зміни.

23. Що показує git log?

Історію комітів поточної гілки. Параметр --oneline скорочує кожен запис до короткого хешу й першого рядка повідомлення.

24. Для чого потрібен .gitignore?

Він задає шаблони файлів, які Git не пропонуватиме додати як нові. Уже відстежуваний файл не перестає відстежуватися лише через появу його назви в .gitignore.

25. Чому .venv не потрібно комітити?

Середовище містить багато згенерованих і платформозалежних файлів. Його створюють повторно за вимогами проєкту, замість перенесення готової копії між комп’ютерами.

26. Для чого потрібний README?

Він пояснює призначення проєкту та дає команди встановлення, запуску й перевірки. Завдяки йому інший користувач може відтворити роботу програми.

27. Що таке type hints?

Це анотації очікуваних типів параметрів, змінних і результатів. Наприклад, Student | None означає запис студента або відсутність результату. Самі анотації не перевіряють типи під час виконання.

28. Для чого програму розділяють на modules?

Щоб частини коду мали зрозуміле призначення і їх було простіше змінювати та перевіряти. У цій роботі перевірка обчислень не потребує введення даних у меню.

29. Що таке business logic?

Це правила обробки даних предметної області. Тут до них належать відбір студентів групи, пошук найбільшого бала та обчислення середнього значення.

30. Чому потрібно відокремлювати application entry point від business logic?

Точка входу керує запуском і взаємодією з користувачем, а логіка виконує операції з даними. Такий поділ дозволяє перевіряти services окремо й надалі використати ті самі функції в іншому інтерфейсі.

12. Висновок

У ході виконання лабораторної роботи створено консольну систему обліку студентів за варіантом №1. Програму організовано за src-layout: модель Student відокремлено від операцій зі списком і консольного меню. Реалізовано всі п’ять операцій завдання, перевірку введення та рейтинг. На контрольних даних отримано середній бал ФЕП-32 89,00, а найкращим визначено студента з балом 95,00. Усі 14 автоматизованих перевірок пройдено. Пакет установлено в editable mode, підготовлено README, pyproject.toml і .gitignore, а зміни збережено трьома комітами Git. Робота дала змогу практично відпрацювати створення середовища, модульну організацію коду та перевірку результатів.